

02 · JavaScript 비동기 프로그래밍

콜백 지옥이 왜 생기고, Promise와 async/await이 그것을 어떻게 푸는가

Node.js는 **싱글 스레드 + 이벤트 루프** 구조입니다. 요청마다 스레드를 만들지 않고 하나의 루프가 전부 처리합니다. 적은 자원으로 많은 동시 연결을 감당할 수 있는 대신, **한 곳에서 오래 멈추면 서버 전체가 멈춥니다.**

그래서 파일·네트워크·DB처럼 기다리는 작업은 반드시 비동기로 처리해야 합니다. 이 장은 그 비동기 코드를 어떻게 읽기 좋게 쓰느냐에 대한 이야기입니다.

학습 목표

- 동기/비동기와 블로킹/논블로킹을 구분해서 설명한다
- 콜백 중첩이 만드는 두 가지 문제를 안다
- Promise의 세 가지 상태와 체이닝을 이해한다
- 순차 실행과 병렬 실행의 차이를 시간으로 설명한다

1. 동기 vs 비동기 — 실행 순서로 확인

말로 설명하는 것보다 출력 순서를 보는 게 빠릅니다. 동기 함수는 끝날 때까지 다음 줄로 못 가고, 비동기 함수는 던져 놓고 바로 넘어갑니다.

02-async/01-sync-vs-async.js

```
/**
 * 동기(sync) vs 비동기(async)
 *
 * 동기 : 앞의 작업이 끝나야 다음 줄이 실행된다. 순서 보장, 대신 그동안 멈춤.
 * 비동기: 작업을 맡기고 바로 다음 줄로 넘어간다. 결과는 나중에 콜백/프로미스로.
 *
 * 실행: node 02-async/01-sync-vs-async.js
 */

function syncFunc() {
  console.log('동기 작업 시작');
  let result = 0;
  for (let i = 0; i < 1000000; i++) result += 1;
  console.log(`동기 작업 완료: ${result}`);
  return result;
}

function asyncFunc() {
  console.log('비동기 작업 시작');
  return new Promise((resolve) => {
    // 실제 비동기 효과를 보기 위해 setTimeout 으로 다음 턴에 넘긴다.
    setTimeout(() => {
      let result = 0;
      for (let i = 0; i < 1000000; i++) result += 1;
      resolve(result);
    }, 1000);
  });
}
```

```

    }, 0);
  });
}

syncFunc(); // 여기서 끝날 때까지 아래로 못 내려감
console.log('다른 작업1');
console.log('다른 작업2');

asyncFunc().then((result) => {
  console.log(`비동기 작업 완료: ${result}`); // 가장 마지막에 출력된다
});

console.log('다른 작업3');
console.log('다른 작업4');

/*
출력 순서
동기 작업 시작
동기 작업 완료: 1000000
다른 작업1
다른 작업2
비동기 작업 시작      <- Promise 생성자 본문은 동기로 실행됨에 주의
다른 작업3
다른 작업4
비동기 작업 완료: 1000000
*/

```

실행

```
node 02-async/01-sync-vs-async.js
```

실행 결과

```

동기 작업 시작
동기 작업 완료: 1000000
다른 작업1
다른 작업2
비동기 작업 시작
다른 작업3
다른 작업4
비동기 작업 완료: 1000000

```

짚고 갈 것

- `new Promise(...)` 생성자 본문은 동기로 실행됩니다. "비동기 작업 시작"이 먼저 찍히는 이유
- `resolve`로 넘긴 값만 나중에 `.then`으로 전달됩니다
- `for (i = 0; ...)`처럼 `let`을 빠뜨리면 암묵적 전역 변수가 되어 다른 코드와 충돌합니다

2. 콜백 지옥 — 무엇이 문제인가

비동기 작업을 순서대로 이으려고 콜백 안에 콜백을 넣으면 두 가지 문제가 생깁니다.

02-async/02-callback-hell.js

```
/**
 * 콜백 지옥 (callback hell)
 *
 * 비동기 작업을 순서대로 이어붙이려고 콜백 안에 콜백을 중첩하면
 * 오른쪽으로 계속 밀리는 코드가 된다. 에러 처리도 각 단계마다 따로 해야 한다.
 * -> 이 문제를 풀려고 나온 것이 Promise, 그리고 async/await 이다.
 *
 * 실행: node 02-async/02-callback-hell.js
 */

const DB = [];

function saveDB(user, callback) {
  DB.push(user);
  console.log(`save ${user.name}`);
  return callback(user);
}

function sendEmail(user, callback) {
  console.log(`send email to ${user.email}`);
  return callback(user);
}

function getResult(user) {
  return `success register ${user.name}`;
}

// 단계가 늘어날수록 들여쓰기가 계속 깊어진다.
function register(user) {
  return saveDB(user, (user) => {
    return sendEmail(user, (user) => {
      return getResult(user);
    });
  });
}

const result = register({ name: 'web', email: 'web@naver.com' });
console.log(result);

// ----- 실제 비동기라면 더 심해진다 -----
console.log(`\n--- 실제 비동기 콜백 중첩 ---`);

const delay = (ms, value, cb) => setTimeout(() => cb(null, value), ms);

delay(100, '1단계', (err, a) => {
  if (err) return console.error(err);
  console.log(a);
});
```

```

delay(100, '2단계', (err, b) => {
  if (err) return console.error(err);
  console.log(b);
  delay(100, '3단계', (err, c) => {
    if (err) return console.error(err); // 에러 처리가 단계마다 반복된다
    console.log(c);
    console.log('완료 (오른쪽으로 계속 밀리는 코드)');
  });
});
});
});

```

실행

```
node 02-async/02-callback-hell.js
```

실행 결과

```

save web
send email to web@naver.com
success register web

```

--- 실제 비동기 콜백 중첩 ---

```

1단계
2단계
3단계
완료 (오른쪽으로 계속 밀리는 코드)

```

짚고 갈 것

- 들여쓰기가 계속 깊어집니다. 단계가 5개만 되어도 화면 밖으로 밀립니다
- 에러 처리가 단계마다 반복됩니다. 한 곳으로 모을 수 없습니다
- 이 두 문제를 풀려고 나온 것이 Promise, 그 위에 문법을 얹은 것이 async/await 입니다

3. Promise — 상태와 체이닝

Promise는 "비동기 작업의 최종 결과를 담는 객체"입니다. 상태는 셋 중 하나입니다.

- **pending** 대기 중
- **fulfilled** 성공 → **resolve(value)** 호출됨 → **.then(value)**로 받음
- **rejected** 실패 → **reject(error)** 호출됨 → **.catch(error)**로 받음

.then().then().catch()로 체이닝하면 콜백 지옥의 중첩이 평평해지고, 에러 처리가 **catch** 한 곳으로 모입니다.

02-async/03-promise.js

```

/**
 * Promise - 비동기 작업의 최종 결과를 담는 객체

```

```

*
* 상태 3가지
*   pending    : 대기 중
*   fulfilled  : 성공 -> resolve(value) 호출됨 -> .then(value) 로 받음
*   rejected   : 실패 -> reject(error) 호출됨 -> .catch(error) 로 받음
*
* .then().then().catch().finally() 로 체이닝하면
* 콜백 지옥의 중첩이 평평하게 퍼지고 에러 처리도 catch 한 곳으로 모인다.
*
* 실행: node 02-async/03-promise.js
*/

```

```
const DB = [];
```

```
function saveDB(user) {
  const oldLength = DB.length;
  DB.push(user);
  console.log(`save ${user.name} to DB`);
  return new Promise((resolve, reject) => {
    if (DB.length > oldLength) resolve(user);
    else reject(new Error('Save DB Error!'));
  });
}
```

```
function sendEmail(user) {
  console.log(`send email to ${user.name}`);
  return new Promise((resolve) => resolve(user));
}
```

```
function getResult(user) {
  return new Promise((resolve) => {
    resolve(`success register ${user.name}`);
  });
}
```

```
function registerByPromise(user) {
  return saveDB(user)
    .then(sendEmail) // 앞 then 의 반환값이 다음 then 의 인자로
    .then(getResult)
    .catch((error) => new Error(error)) // 체인 어디서 에러가 나든 여기로
    .finally(() => console.log('완료 (성공/실패 무관하게 항상 실행)'));
}
```

```
const myUser = { name: 'wizard', email: 'wizard113@nambu.com' };
registerByPromise(myUser).then(console.log);
```

```
// ----- 여러 프로미스를 한 번에 다루기 -----
```

```
const delay = (ms, value) => new Promise((r) => setTimeout(() => r(value), ms));
```

```

// all      : 전부 성공해야 성공. 하나라도 실패하면 즉시 reject
// allSettled: 실패해도 끝까지 기다리고 각각의 상태를 돌려줌
// race     : 가장 먼저 끝난 하나 (타임아웃 구현에 사용)
// any      : 가장 먼저 성공한 하나

```

```

Promise.all([delay(100, 'A'), delay(200, 'B'), delay(50, 'C')]).then((r) =>
  console.log('\nPromise.all      : ', r) // [ 'A', 'B', 'C' ] - 입력 순서 유지
);

Promise.allSettled([delay(50, 'OK'), Promise.reject(new Error('실패'))]).then((r) =>
  console.log('Promise.allSettled:', r.map((x) => x.status))
);

Promise.race([delay(100, '느림'), delay(30, '빠름')]).then((r) =>
  console.log('Promise.race      : ', r) // '빠름'
);

```

실행

```
node 02-async/03-promise.js
```

실행 결과

```

save wizard to DB
send email to wizard
완료 (성공/실패 무관하게 항상 실행)
success register wizard
Promise.race      : 빠름
Promise.allSettled: [ 'fulfilled', 'rejected' ]

Promise.all      : [ 'A', 'B', 'C' ]

```

참고 할 것

- 앞 **then**의 반환값이 다음 **then**의 인자로 넘어갑니다
- **.catch**에서 에러를 반환만 하면 체인이 성공으로 이어집니다. 다시 **throw** 해야 호출한 쪽이 실패를 압니다
- **.finally**는 성공/실패와 무관하게 항상 실행됩니다 (로딩 스피너 끄기 등)
- **Promise.all**은 입력 순서대로 결과를 돌려줍니다. 먼저 끝난 순서가 아닙니다

네 가지 조합 메서드

메서드	동작	언제 쓰나
Promise.all	전부 성공해야 성공. 하나라도 실패하면 즉시 reject	여러 API를 동시에 부르고 전부 필요할 때
Promise.allSettled	실패해도 끝까지 기다리고 각각의 상태를 반환	일부 실패해도 나머지 결과가 필요할 때
Promise.race	가장 먼저 끝난 하나	타임아웃 구현
Promise.any	가장 먼저 성공한 하나	여러 미러 중 빠른 곳에서 받기

4. async / await — 그리고 순차 vs 병렬

`async` 함수는 반환값을 항상 Promise로 감쌉니다. `await`은 Promise가 끝날 때까지 그 자리에서 기다립니다. 에러는 `try/catch`로 잡습니다.

이 절의 핵심은 순차와 병렬의 차이입니다. 서로 의존하지 않는 작업을 하나씩 `await`하면 시간이 그대로 더해집니다. 출력의 소요 시간을 직접 비교해 보세요.

02-async/04-async-await.js

```
/**
 * async / await - Promise 를 동기 코드처럼 읽히게 만드는 문법
 *
 *   async 함수는 반환값을 항상 Promise 로 감싼다.
 *   await 는 Promise 가 끝날 때까지 그 자리에서 기다린다. (async 함수 안에서만 사용)
 *   에러는 try/catch 로 잡는다.
 *
 * 실행: node 02-async/04-async-await.js
 */

// async 가 붙으면 return 'hello' 도 Promise.resolve('hello') 가 된다.
async function func1() {
  return 'hello';
}
func1().then(console.log); // hello

// 위 async 함수는 아래 Promise 함수와 동일하다.
function func2() {
  return new Promise((resolve) => resolve('hello'));
}
func2().then(console.log); // hello

// await 로 받으면 then 없이 값을 바로 꺼낼 수 있다.
async function func3() {
  const name = await func1();
  console.log('await 로 받은 값:', name);
}
func3();

// ----- 순차 실행 -----
function waitOneSecond(msg) {
  return new Promise((resolve) => setTimeout(() => resolve(msg), 300));
}

async function countOneToFive() {
  console.log('\n--- 순차 실행 (앞이 끝나야 다음) ---');
  console.time('순차');
  for (let i = 0; i < 5; i++) {
    const result = await waitOneSecond(`${i + 1}번째 대기 중...`);
    console.log(result);
  }
  console.timeEnd('순차'); // 약 1500ms
```

```

}

// ----- 병렬 실행 -----
// 서로 의존하지 않는 작업은 Promise.all 로 동시에 던져야 빠르다.
async function countParallel() {
  console.log('\n--- 병렬 실행 (동시에 던지고 한 번에 기다림) ---');
  console.time('병렬');
  const tasks = [];
  for (let i = 0; i < 5; i++) tasks.push(waitOneSecond(`${i + 1}번째`));
  const results = await Promise.all(tasks);
  console.log(results);
  console.timeEnd('병렬'); // 약 300ms
}

// ----- 에러 처리 -----
async function mayFail(shouldFail) {
  if (shouldFail) throw new Error('의도적인 에러');
  return '성공';
}

async function main() {
  await countOneToFive();
  await countParallel();

  console.log('\n--- 에러 처리 ---');
  try {
    console.log(await mayFail(false));
    console.log(await mayFail(true)); // 여기서 throw
  } catch (err) {
    console.error('catch 에서 잡음:', err.message);
  } finally {
    console.log('finally 는 항상 실행');
  }
}

main();

```

실행

```
node 02-async/04-async-await.js
```

실행 결과

```

--- 순차 실행 (앞이 끝나야 다음) ---
hello
hello
await 로 받은 값: hello
1번째 대기 중...

```

```

2번째 대기 중...
3번째 대기 중...
4번째 대기 중...
5번째 대기 중...
순차: 1.528s

--- 병렬 실행 (동시에 던지고 한 번에 기다림) ---
[ '1번째', '2번째', '3번째', '4번째', '5번째' ]
병렬: 305.121ms

--- 에러 처리 ---
성공
catch 에서 잡음: 의도적인 에러
finally 는 항상 실행

```

짚고 갈 것

- 순차 실행 약 1500ms vs 병렬 실행 약 300ms — 같은 작업인데 5배 차이
- 서로 의존하지 않으면 **Promise.all**로 묶으세요
- **await**은 **async** 함수 안에서만 씁니다 (또는 ESM의 top-level await)
- **async** 함수의 반환값도 Promise입니다. 호출한 쪽에서도 **await**이나 **then**이 필요합니다

안티패턴 — 반복문 안의 await

```

// 느림 - 300ms x 5 = 1500ms
for (const id of ids) {
  await fetchUser(id);
}

// 빠름 - 약 300ms
await Promise.all(ids.map(fetchUser));

// 단, 앞 결과가 다음 요청에 필요하다면 순차가 맞습니다
const user = await fetchUser(id);
const posts = await fetchPosts(user.id); // user 가 있어야 가능

```

정리

방식	장점	단점
콜백	별도 문법이 필요 없다	중첩이 깊어지고 에러 처리가 흩어진다
Promise	체이닝으로 평평해지고 에러가 한곳에 모인다	then 이 많아지면 여전히 읽기 어렵다
async/await	동기 코드처럼 위에서 아래로 읽힌다	무심코 순차 실행이 되기 쉽다